



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**título del TFG
Documentación Técnica**



Presentado por nombre alumno
en Universidad de Burgos — 2 de junio
de 2025

Tutor: nombre tutor

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	iv
Apéndice A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Planificación Temporal	1
A.3. Estudio de Viabilidad	25
Apéndice B Especificación de Requisitos	29
B.1. Introducción	29
B.2. Objetivos Generales del Proyecto	29
B.3. Catálogo de Requisitos	30
B.4. Especificación de requisitos	34
Apéndice C Especificación de diseño	37
C.1. Introducción	37
C.2. Diseño de datos	37
C.3. Diseño arquitectónico	37
C.4. Diseño procedimental	37
Apéndice D Documentación técnica de programación	39
D.1. Introducción	39
D.2. Estructura de directorios	39
D.3. Manual del programador	39

D.4. Compilación, instalación y ejecución del proyecto	39
D.5. Pruebas del sistema	39
Apéndice E Documentación de usuario	41
E.1. Introducción	41
E.2. Requisitos de usuarios	41
E.3. Instalación	41
E.4. Manual del usuario	41
Apéndice F Anexo de sostenibilización curricular	43
F.1. Introducción	43
F.2. Contribución del Proyecto a los Objetivos de Desarrollo Sostenible (ODS)	43
F.3. Competencias de Sostenibilidad Adquiridas y Aplicadas	45
F.4. Reflexión Final	46
Bibliografía	47

Índice de figuras

A.1. Ejemplo de la estructura de ramas utilizada en el repositorio de GitHub del proyecto, mostrando diferentes líneas de desarrollo para funcionalidades clave.	3
A.2. histograma de commits realizados	3
A.3. Ejemplo del tablero Kanban en Zube utilizado para la gestión de tareas de un sprint.	4
A.4. Analíticas de espacio de trabajo ofrecidas por Zube para el seguimiento de proyectos.	4
A.5. Gráfico Burndown del Sprint 5, ilustrando la reducción de tareas pendientes durante la revisión de las clases MVC.	5
A.6. GráficoBurnup: SP4 - Implantación de la aplicación usando MVC.	5
A.7. Burndown sprint 0	6
A.8. Burndown sprint 1	7
A.9. Burndown sprint 2	9
A.10.Burndown sprint 3	11
A.11.Formato inicial de archivo XML para la configuración de los bloques y su jerarquía.	14
A.12.Burndown sprint 4	15
A.13.Burndown sprint 5	17
A.15.Burndown sprint 6	20
A.14.Depuración de las conexiones desde la consola de unity cuando se ejecutaba el programa.	21

Índice de tablas

B.1. Descripción del Actor A01 - Estudiante.	35
B.2. CU01 - Gestionar Interfaz de Programación.	36

Apéndice A

Plan de Proyecto Software

A.1. Introducción

El Plan de Proyecto Software detalla los aspectos clave de la planificación, gestión y viabilidad del desarrollo del prototipo del Laboratorio Virtual de Programación y Robótica. Este documento tiene como objetivo establecer un marco de referencia para la ejecución del Trabajo Fin de Grado (TFG), asegurando que el prototipo se desarrolle de manera eficiente y cumpla con los objetivos establecidos.

La planificación se ha enfocado en un desarrollo iterativo, aplicando principios de metodologías ágiles (SCRUM) para dividir el proyecto en tareas, sprints y entregables incrementales. Esto ha permitido una adaptación continua a los desafíos técnicos y una validación temprana de las funcionalidades principales.

Este apéndice también incluye un análisis de la viabilidad del proyecto, considerando los recursos humanos y materiales involucrados, los costes asociados estimados para el desarrollo del prototipo y una evaluación básica de los aspectos legales relevantes. El objetivo es demostrar que el proyecto, dentro de su alcance como TFG, es factible y se ha gestionado de manera responsable.

A.2. Planificación Temporal

La ejecución de este Trabajo Fin de Grado se ha gestionado mediante una adaptación de la metodología ágil **SCRUM**. Esta elección metodológica se fundamenta en la necesidad de un desarrollo iterativo e incremental,

permitiendo flexibilidad ante los desafíos técnicos y la evolución de los requisitos inherentes a un proyecto de prototipado y desarrollo de software. La planificación se ha estructurado en ciclos de trabajo o *sprints*, de duración variable (generalmente entre una y tres semanas), con el objetivo de producir incrementos funcionales del laboratorio virtual y facilitar una validación continua del progreso.

Herramientas y Artefactos de Planificación

Para llevar a cabo la planificación temporal y el seguimiento del proyecto, se han utilizado diversas herramientas y artefactos metodológicos.

- **GitHub:** Ha funcionado como el sistema de control de versiones (Git) y el repositorio centralizado para todo el código fuente. Una estrategia fundamental en su uso ha sido la **gestión mediante ramas** (branches) siendo un total de 13 las creadas. Cada nueva funcionalidad significativa o la corrección de errores complejos se abordó en una rama separada ejemplos de ramas utilizadas durante el desarrollo se pueden observar en la Figura A.1).

Este enfoque permitió aislar el desarrollo de estas características, experimentar con soluciones sin afectar la estabilidad de la rama principal (**master**), y facilitó la revisión e integración controlada de los cambios. El historial de *commits* (ver figura A.2) en cada rama ha proporcionado una trazabilidad detallada de la evolución específica de cada módulo o funcionalidad. Adicionalmente, las funcionalidades de *issues* de GitHub se emplearon para el registro y seguimiento de tareas, así como para la gestión de errores o mejoras identificadas durante el desarrollo.

GitHub ha funcionado como el sistema de control de versiones (Git) y el repositorio centralizado para todo el código fuente; las ramas permitieron gestionar el desarrollo de forma aislada y los *commits* proporcionaron trazabilidad. Las *issues* de GitHub se emplearon para el registro de tareas y la gestión de errores.

Branch	Updated	Check status	Behind / Ahead	Pull request
Testslintity	1 hour ago		31 3	
CodeExecution	3 days ago		31 0	#67
StackSnappingBlocks	last week		43 0	#63
SnappingBlocks	3 weeks ago		47 0	#60
HightlightConnection	last month		58 0	#58
ConnectionsMVC	2 months ago		63 0	#56
BlockConnections	2 months ago		67 0	#55
BlockJoins	3 months ago		157 1	
Connections	3 months ago	0 / 1	168 0	#39
Imagen	3 months ago	1 / 1	168 0	#38
DragAndDrop	3 months ago	1 / 1	173 0	#37
Pruebas	4 months ago		249 0	

Figura A.1: Ejemplo de la estructura de ramas utilizada en el repositorio de GitHub del proyecto, mostrando diferentes líneas de desarrollo para funcionalidades clave.

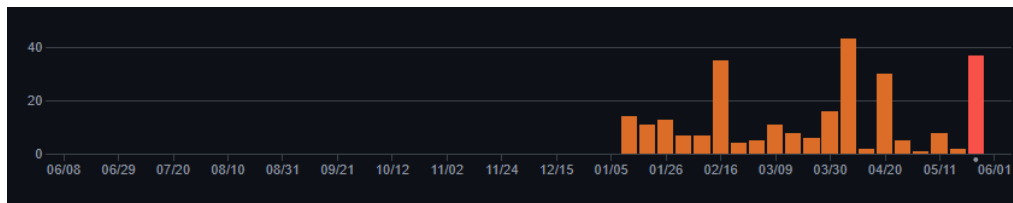


Figura A.2: histograma de commits realizados

Complementariamente, la plataforma de gestión de proyectos **Zube** [2], integrada con GitHub, fue utilizada para la organización y visualización del trabajo. Sus funcionalidades principales empleadas han sido:

- **Gestión de Tareas en Tableros Kanban:** Permitiendo la visualización del flujo de trabajo de cada tarea, desde su planificación (Inbox) hasta su finalización (Done), incluyendo estados intermedios. En la Figura A.3 se muestra un ejemplo de este tablero.
- **Planificación y Seguimiento de Sprints:** Facilitando la definición de objetivos y el conjunto de tareas (historias de usuario con sus respectivos puntos de historia) a abordar en cada iteración.

- **Métricas Ágiles y Analíticas del Espacio de Trabajo:** Aunque para un proyecto individual la aplicación estricta de todas las métricas de equipo puede variar, Zube ofrece diversas analíticas (ver Figura A.4) que han servido como referencia para evaluar el progreso general.

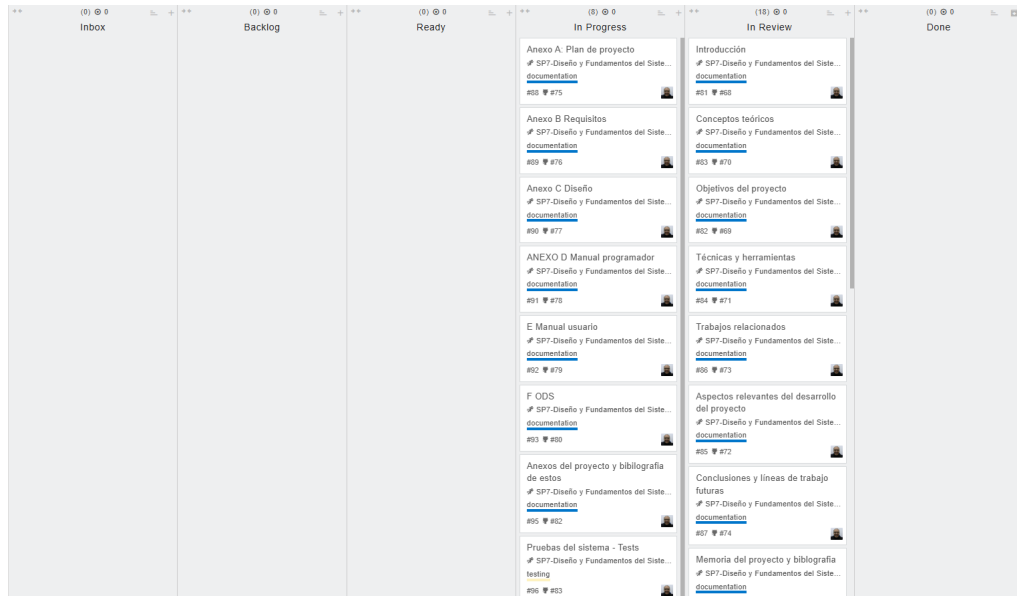


Figura A.3: Ejemplo del tablero Kanban en Zube utilizado para la gestión de tareas de un sprint.

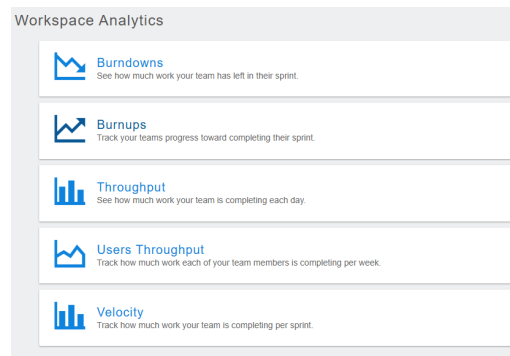


Figura A.4: Analíticas de espacio de trabajo ofrecidas por Zube para el seguimiento de proyectos.

Entre estas métricas, destacan conceptualmente:

- **Gráficos Burndown:** Útiles para visualizar el trabajo restante en un sprint como el que se muestra en la figura A.5.
- **Gráficos Burnup:** Permiten rastrear el progreso acumulado tal y como se puede ver en la figura A.6.
- **Velocity (Velocidad):** Indicador del trabajo completado por sprint, ayudando a calibrar la carga de trabajo futura.

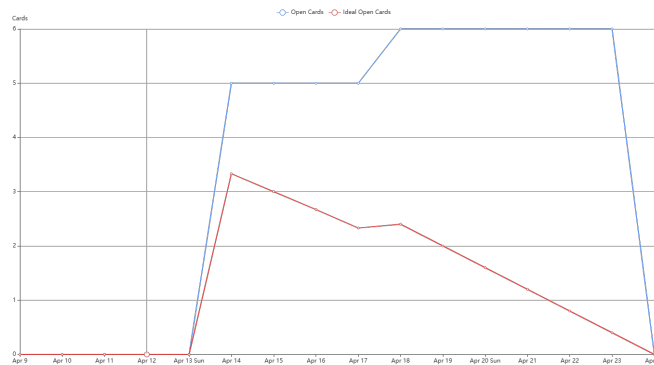


Figura A.5: Gráfico Burndown del Sprint 5, ilustrando la reducción de tareas pendientes durante la revisión de las clases MVC.

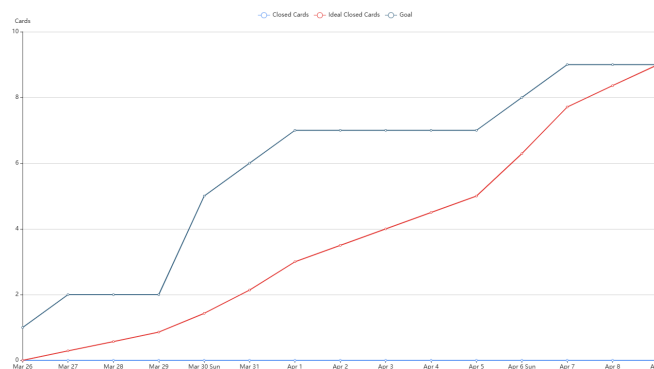


Figura A.6: Gráfico Burnup: SP4 - Implantación de la aplicación usando MVC.

La asignación de puntos de historia y su seguimiento en Zube, aunque adaptado a un contexto individual, permitió una estimación cualitativa del esfuerzo y una evaluación continua del avance en las distintas fases del proyecto.

Hitos Principales del Proyecto

Se definieron los siguientes hitos clave para marcar puntos significativos en el desarrollo del prototipo:

Sprint 0: Puesta en marcha, Configuración del Entorno e Investigación Inicial

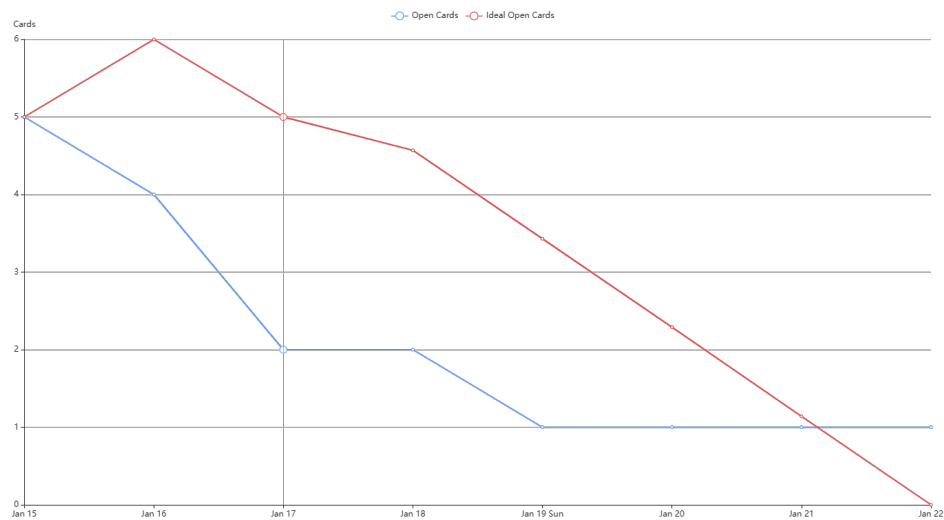


Figura A.7: Burndown sprint 0

- **Duración:** Semana 1 (del 15 al 22 de Enero de 2025).
- **Objetivo Principal:** Establecer las bases del proyecto, configurar el entorno de desarrollo y realizar una investigación exhaustiva de las herramientas y librerías clave, con especial foco en UBlockly.
- **Contexto:** Tras las reuniones iniciales con el tutor para definir el alcance y los objetivos del TFG, este primer sprint se centró en preparar toda la infraestructura técnica y conceptual necesaria para iniciar la implementación y la configuración de las diferentes partes del repositorio en el que se iba a trabajar.
- **Tareas Principales y Resultados:**
 - Instalación y configuración de las herramientas de desarrollo: Unity (Versión 6000.0.2f1), C#, Visual Studio, Git y Zube.

- Creación del repositorio del proyecto en GitHub e integración inicial con Zube para la gestión de tareas.
 - Análisis profundo de la arquitectura y componentes del repositorio UBlockly: se estudiaron sus clases `BlockModel`, `ConnectionModel`, el sistema de ejecución (`CSharpRunner`, `Cmdtors`) y su integración con Unity UI.
 - Esbozo inicial de la estructura de la interfaz de usuario principal (`UICanvasView`) en Unity, incluyendo la disposición de los paneles para categorías, toolbox y área de codificación.
 - Investigación sobre la representación de robots educativos como LEGO WeDo 2.0 en entornos 3D y los principios de la programación visual estilo Scratch.
- **Hito Alcanzado:** Entorno de desarrollo plenamente operativo. Comprensión clara de la base técnica de UBlockly para su adaptación.

Sprint 1: Resumen de Objetivos y Comparativa de Trabajos Relacionados

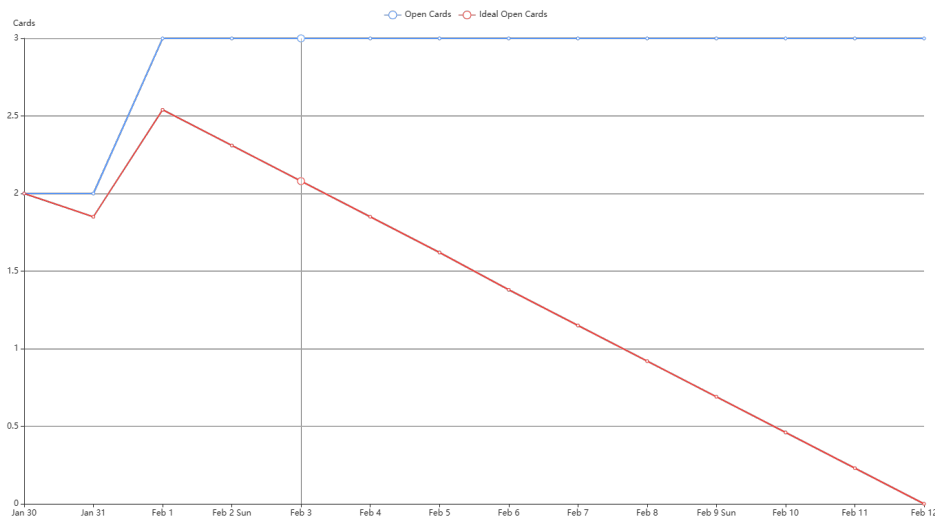


Figura A.8: Burndown sprint 1

- **Duración;** Del 30 de Enero de 2025 al 12 de Febrero de 2025.

- **Objetivo Principal:** Consolidar los objetivos del TFG con un enfoque en la tríada LEGO-Scratch-Unity, realizar un estudio comparativo de robots y paletas de trabajo, y avanzar en la documentación técnica y conceptual.
- **Contexto:** Tras la puesta en marcha inicial del proyecto (Sprint 0), este sprint se centró en definir con mayor precisión el alcance técnico y teórico, así como en establecer una base sólida para la comparativa con trabajos existentes y la estructuración de la memoria.
- **Tareas Clave Realizadas :**
 - Elaboración de un resumen de los objetivos del proyecto, con un enfoque específico en la integración y adaptación de los paradigmas de LEGO, Scratch y Unity.
 - Creación y refinamiento de la lista de acrónimos a utilizar en la documentación.
 - Sistematización del etiquetado de *issues* y del flujo de *commits* en GitHub para facilitar el seguimiento del tutor.
 - Realización de un estudio comparativo de diferentes kits de robótica educativa y sus entornos de programación para concretar el modelo de robot virtual a simular y sus propiedades fundamentales.
 - Investigación de las paletas de trabajo y tipos de bloques utilizados tanto en Scratch (como estándar de código abierto para la programación visual educativa) como en entornos LEGO, identificando qué estructuras son más adecuadas para el tipo de robot objetivo.
 - Definición detallada de las propiedades del robot virtual que se iban a implementar y simular en el prototipo.
 - Ajuste de la terminología interna del proyecto (ej. cambiar etiqueta formación a learning).
 - Concreción de los objetivos educativos del laboratorio, alineándolos con conceptos de accesibilidad, herramientas de calidad y desarrollo sostenible (ODS).
 - Inicio de la sección de Trabajos Relacionados de la memoria, incluyendo la preparación de una tabla comparativa con las plataformas analizadas.
 - Definición del alcance inicial de la gamificación (aunque su implementación completa se planificara para fases posteriores).

- Delimitación conceptual del Laboratorio Virtual frente a un entorno virtual genérico.
 - Avance en la documentación contextualizada del proyecto, asegurando que cada concepto teórico estuviera bien anclado y justificado.
 - Documentación de las decisiones de diseño y técnicas tomadas hasta la fecha.
- **Hito Alcanzado:** Objetivos del TFG refinados y claramente enfocados. Primera versión del análisis comparativo de plataformas y robots. Estructura inicial de la memoria del TFG definida y con contenido preliminar en secciones clave. Protocolos de gestión de código y tareas establecidos.

Sprint 2: Creación de un Prototipo de Prueba y Avance Documental

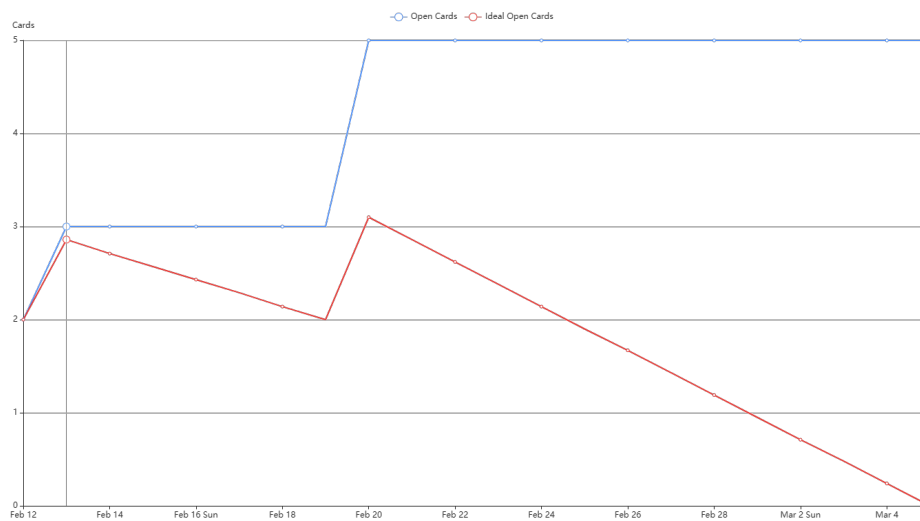


Figura A.9: Burndown sprint 2

- **Duración:** Del 12 de Febrero de 2025 al 5 de Marzo de 2025.
- **Objetivo Principal:** Desarrollar una primera prueba funcional del robot virtual (inspirado en WeDo 2.0) capaz de moverse en el entorno 3D de Unity, crear un APK para pruebas en dispositivo, y avanzar

significativamente en la documentación y estructuración de la memoria del TFG.

- **Contexto:** Sobre la base de los objetivos y la investigación del sprint anterior, este sprint se enfocó en materializar un primer prototipo ejecutable y en consolidar la estructura de la memoria, abordando aspectos técnicos y de contenido.
- **Tareas Clave Realizadas:**
 - Diseño y desarrollo de una prueba inicial de movimiento para el robot virtual WeDo 2.0 en Unity.
 - Generación de un archivo APK (Android Package Kit) para la instalación y prueba del prototipo en un dispositivo Android físico.
 - Planificación de tareas y organización de *issues* en GitHub, incluyendo su correcta categorización, asignación de salidas esperadas y uso de hashtags para referenciar *commits* asociados.
 - Elaboración de un resumen ejecutivo del trabajo y la descripción del proyecto para la memoria.
 - Refinamiento de la estructura del repositorio, excluyendo archivos innecesarios.
 - Redacción de la sección About (Acerca de) del repositorio del proyecto.
 - Revisión y selección de los Objetivos de Desarrollo Sostenible (ODS 4 y ODS 10) más relevantes para el TFG, justificando su elección.
 - Búsqueda y análisis de artículos y recursos sobre robótica 3D y simulación para enriquecer el marco teórico.
 - Consolidación del formato para las citas bibliográficas (uso de archivos *.bib*, formato autor-año) y referencias.
 - Clarificación conceptual sobre la distinción entre un entorno virtual genérico y la simulación de un robot con comportamiento específico.
 - Revisión y ajuste de los objetivos funcionales y no funcionales del proyecto en base a los avances.
 - Revisión y mejora de la introducción y la sección de objetivos de la memoria del TFG.

- Definición de los componentes de la prueba en 3D con el robot virtual, incluyendo:
 - Modelado 3D básico del robot.
 - Diseño de la interfaz de usuario (UI) inicial para la interacción con los bloques y la simulación.
 - Elaboración del resumen y título del trabajo para ser incluidos en la memoria.
 - Se configuro la herramienta de latex para la generación de los pdfs.
- **Hito Alcanzado:** Primera versión de un prototipo de aplicación donde se podía ver los distintos paneles y la proyección de las categorías de los bloques. Avance sustancial en la redacción de la introducción, objetivos, y marco teórico de la memoria. Gestión de tareas y versionado en GitHub/Zube consolidada.

Sprint 3: Reevaluación y Desarrollo del Prototipo Funcional del Robot

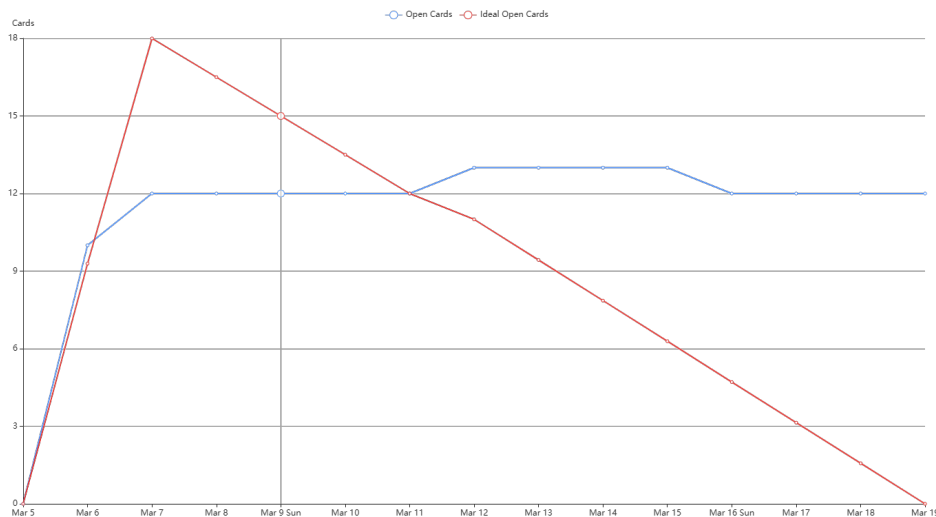


Figura A.10: Burndown sprint 3

- **Duración:** Del 5 de Marzo de 2025 al 19 de Marzo de 2025.

- **Objetivo Principal:** Superar las limitaciones de la simulación básica inicial (basada en la ejecución directa de UBlockly que no permitía una exportación APK satisfactoria) y desarrollar un prototipo funcional donde el movimiento de un robot virtual 3D fuese visible, controlable a través de los bloques, y empaquetable como una aplicación Android.
- **Contexto y Reevaluación:** Las pruebas realizadas en sprints anteriores con la estructura de ejecución directa de UBlockly revelaron que, si bien permitían una lógica de movimiento básica (ej. mover una esfera en el escenario de Unity), no se adaptaban adecuadamente para generar una aplicación Android (.apk) donde un robot 3D claramente identificable ejecutara las acciones programadas por bloques. Esto motivó una **reestructuración del enfoque de la simulación y la interacción**, orientándolo hacia una mayor integración entre la lógica de los bloques de UBlockly (adaptada) y un sistema de control del robot más robusto y visualmente representativo en Unity.
- **Tareas Clave Realizadas:**
 - **Desarrollo del Robot Virtual y Pruebas de Movimiento:**
 - Construcción y/o refinamiento de la versión física del robot con piezas de Lego reales (modelo 3D) para la simulación en Unity.
 - Implementación y depuración de la lógica para la traducción de bloques en una ejecución del movimiento, asegurando que las instrucciones del bloque se convirtieran en acciones concretas del robot virtual
 - Definición y ejecución de la definición prueba en el entorno 3D con el robot virtual para validar su comportamiento básico.
 - **Funcionalidad de Bloques (Movimiento y Eventos):**
 - Desarrollo del Bloque mover - inclusión inputs, asegurando que el bloque de movimiento pudiera aceptar y procesar correctamente los valores de entrada (ej. número de pasos). Se trabajó en entender la estructura de JSON de definición de bloques de UBlockly para adaptarlo al XML con el que se iba a trabajar la configuración de cada tipo de bloque. Se especifican los argumentos que lleva cada bloque (ver A.11) que permiten definir su jerarquía.
 - Se inició la implementación de la funcionalidad para Arrastrar y soltar bloques mover y eventos, permitiendo la composición básica de programas visuales.

- Ajuste del "Bloque mover - posicionar las etiquetas" para una correcta visualización de sus componentes internos.
- Trabajo en el Bloque eventos - para incluir campos imagen (bandera verde) y posicionar las etiquetas, mejorando la configuración y apariencia del bloque de inicio de eventos.
- Se empezó a desarrollar de la funcionalidad para soltar bloques en el área de codificación .
- Se pautaron las bases para la creación y gestión de crear conexiones entre bloques para los bloques de movimiento y evento.

- **Generación y Pruebas en Dispositivo Android:**

- Creación y configuración del proyecto para generar un archivo APK (Crear la apk de prueba para su ejecución) para permitir pruebas directas en dispositivos Android.

- **Gestión del Contenido Visual y Documentación:**

- Ajustar tamaño bloques al contenido para mejorar la presentación visual y el layout dinámico de los bloques según sus campos e inputs .
- Grabación de video robot real de demostración del prototipo funcional del robot para documentación y seguimiento del progreso.

- **Hito Alcanzado:** Se dio forma al prototipo funcional con un robot físico que ejecutaba secuencias de bloques de movimiento iniciadas por un bloque de evento que marcan la base para el prototipo a desarrollar. Se logró la capacidad de generar un APK funcional para pruebas en dispositivo, superando las limitaciones anteriores. Se define además que las versiones deberán ser en 32 y 64 bits por incompatibilidad del Tutor para ponerlo a prueba.

```

<Block>
  <Type>add_to_x</Type>
  <Label>sumar a x %1</Label>
  <SpriteName>Stack_block_grey</SpriteName>
  <connections>
    <top>true</top>
    <bottom>true</bottom>
  </connections>
  <args>
    <arg type="label" value="sumar a x" />
    <arg type="input" name="VALUE" inputType="number" default="5" />
  </args>
</Block>

<Block>
  <Type>subtract_to_x</Type>
  <Label>restar a x %1</Label>
  <SpriteName>Stack_block_grey</SpriteName>
  <connections>
    <top>true</top>
    <bottom>true</bottom>
  </connections>
  <args>
    <arg type="label" value="restar a x" />
    <arg type="input" name="VALUE" inputType="number" default="5" />
  </args>
</Block>

<Block>
  <Type>motion_goto</Type>
  <Label>ir a x:%1 y:%2</Label>
  <SpriteName>Stack_block_grey</SpriteName>
  <connections>
    <top>true</top>
    <bottom>true</bottom>
  </connections>
  <args>
    <arg type="label" value="ir a x:" />
    <arg type="input" name="X" inputType="number" default="0" />
    <arg type="label" value="y:" />
    <arg type="input" name="Y" inputType="number" default="0" />
  </args>
</Block>

```

Figura A.11: Formato inicial de archivo XML para la configuración de los bloques y su jerarquía.

Sprint 4: Reestructuración Arquitectónica e Implementación del Patrón MVC

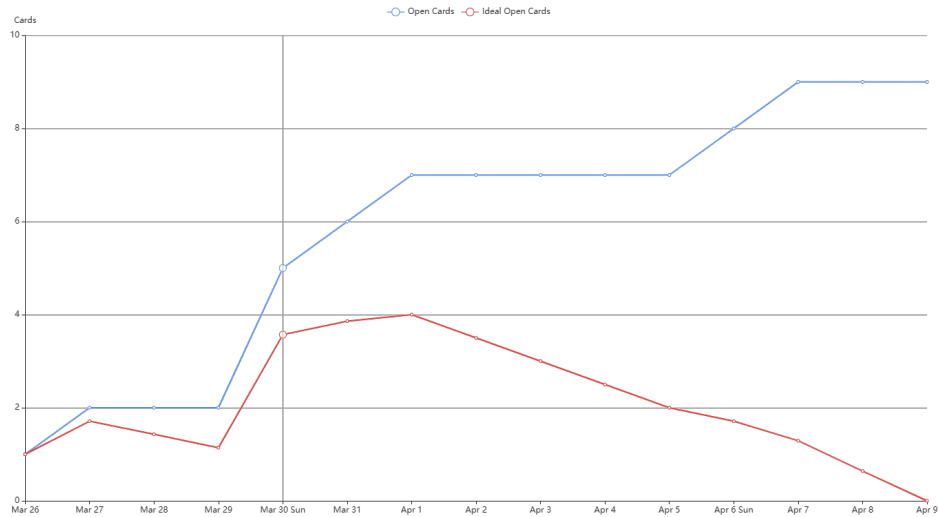


Figura A.12: Burndown sprint 4

- **Duración Estimada:** Del 26 de marzo de 2025 al 9 de abril de 2025.
- **Objetivo Principal:** Reestructurar la arquitectura del proyecto para implementar de forma explícita el patrón Modelo-Vista-Controlador (MVC). El objetivo era lograr una separación clara de responsabilidades que permitiera modificar o extender componentes del Modelo, la Vista o los Controladores de forma más independiente, mejorando la mantenibilidad y escalabilidad del sistema.
- **Contexto y Motivación:** Tras el desarrollo inicial de funcionalidades en sprints anteriores (Sprint 3), se identificó la necesidad de una arquitectura más robusta para gestionar la creciente complejidad, especialmente en la interacción entre la lógica de los bloques (UBlockly), la interfaz de usuario de Unity (Canvas) y la simulación del robot. El patrón MVC fue seleccionado por su probada eficacia en la organización de este tipo de sistemas.
- **Tareas Clave Realizadas y Resultados:**
 - **Análisis y Diseño MVC Aplicado al Proyecto:**

- Identificación de las clases existentes que correspondían a cada capa del patrón MVC (Modelo, Vista, Controlador).
- Diseño de las nuevas clases Controlador necesarias para orquestar las interacciones y la lógica de la aplicación.
- **Refactorización y Creación de Componentes Modelo:**
 - Consolidación de las clases que representan el estado y la lógica de negocio pura (ej. `BlockModel`, `ConnectionModel`, `WorkspaceModel` heredados y adaptados de `UBlockly`) como la capa Modelo, asegurando su independencia de la interfaz de Unity.
- **Refactorización y Creación de Componentes Vista:**
 - Definición y/o ajuste de las clases `View` (ej. `BlockView`, `InputView`, `UICanvasView`) para que su única responsabilidad fuera la representación visual de los datos del Modelo y la captura de eventos de UI, delegando la lógica a los Controladores.
 - Se revisaron los prefabs de Unity para asegurar que la jerarquía de `GameObjects` reflejara esta separación.
- **Implementación de Clases Controlador:**
 - Creación e implementación de los controladores principales, como:
 - ◊ `AppController`: Orquestador general de la aplicación.
 - ◊ `WorkspaceController`: Encargado de las modificaciones al `WorkspaceModel`.
 - ◊ `ExecutionController`: Gestión de la ejecución del programa de bloques.
 - ◊ `InputController`: Manejo de la entrada de datos en los campos de los bloques.
 - ◊ `CategoryController`: Gestión de la selección y muestra de categorías de bloques.
 - Establecimiento de los mecanismos de comunicación entre Controladores y las capas de Modelo y Vista (principalmente mediante el patrón Observador y llamadas a métodos específicos).
- **Eliminación de Clases y Código Obsoleto:** Se identificaron y eliminaron clases o fragmentos de código que resultaron redundantes o incompatibles con la nueva arquitectura MVC, simplificando la base del código. .

- **Hito Alcanzado/Resultado del Sprint:** El proyecto fue transformado exitosamente para operar bajo una arquitectura MVC. Se definieron y crearon los principales controladores, separando la lógica de la presentación y el estado. Esta reestructuración sentó una base mucho más sólida, modular y mantenible para el desarrollo de las funcionalidades subsiguientes y para la futura expansión del laboratorio virtual.

Sprint 5: Revisión Profunda de MVC, Interacción de Bloques y Layout Visual

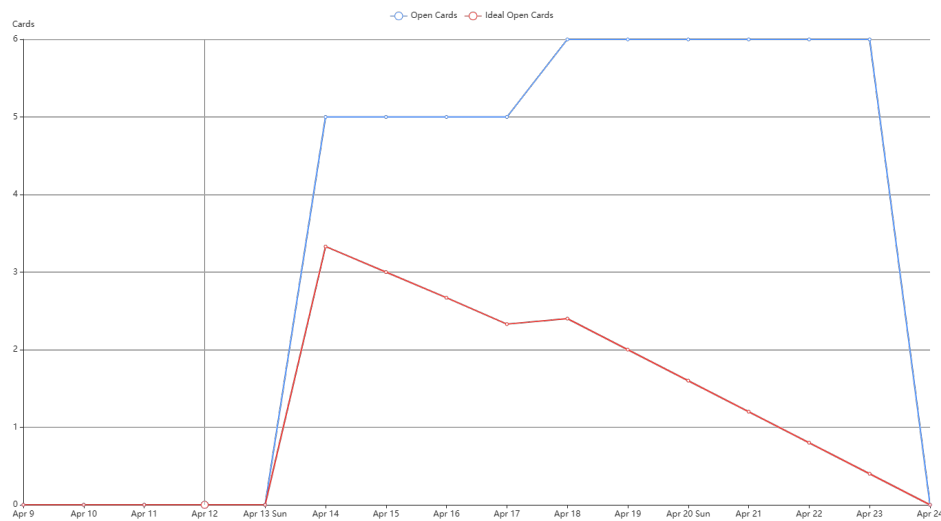


Figura A.13: Burndown sprint 5

- **Duración:** Del 9 de abril de 2025 al 24 de abril de 2025.
- **Objetivo Principal:** Refinar la implementación MVC, abordar los complejos desafíos de la detección de conexiones y el posicionamiento de bloques entre diferentes paneles de Unity UI, y comenzar a preparar la escena de simulación del robot.
- **Contexto y Motivación:** Con la estructura MVC base implementada en el sprint anterior, este sprint se centró en hacer que la interacción con los bloques (arrastré, snap, posicionamiento) funcionara de manera robusta y visualmente coherente, enfrentando las particularidades

del sistema de coordenadas de Unity y el funcionamiento interno de UBlockly. Se detectó que la interacción entre los distintos paneles de la UI (`Toolbox`, `DragLayer`, `CodingArea`) y sus sistemas de coordenadas relativos eran una fuente importante de complejidad para la lógica de snap y el layout visual.

■ **Tareas Clave Realizadas y Resultados:**

- **Refinamiento de la Interacción Modelo-Vista-Controlador:**

- Revisión y ajuste del código de los intérpretes y generadores de UBlockly para asegurar su correcta adecuación al patrón MVC y su interacción con los nuevos controladores.
- Consolidación de la lógica en los controladores `BlockDragController` y `BlockConnectionController` para la gestión del arrastre de bloques, la detección de posibles conexiones y la interacción con el `WorkspaceModel` a través del `WorkspaceController`.

- **Desafíos en la Detección de Conexiones y Layout:**

- Se inició un trabajo intensivo en la lógica de detección de conexiones (`PreviousConnection`, `NextConnection`, `InputValue`). Este esfuerzo reveló la complejidad de gestionar las posiciones relativas de los bloques cuando se mueven entre paneles con diferentes sistemas de coordenadas (Canvas local vs. Screen space, etc.).
- Desarrollo de un **depurador visual/herramienta de log** para monitorizar en tiempo real las conexiones registradas en la base de datos de conexiones (`BlockConnectionDB`) y el estado de las `ConnectionModel`. Esta herramienta fue crucial para diagnosticar y resolver errores de desajuste, conexiones incorrectas o bloques que no se encontraban para el snap (A.14)

- **Posicionamiento y Comportamiento de Arrastre de Bloques:**

- Se realizaron múltiples iteraciones para asegurar que, al iniciar el arrastre desde la `Toolbox`, el bloque clonado se posicionara correctamente bajo el cursor en la capa de arrastre (`DragLayer`) y siguiera la trayectoria del ratón de forma precisa.
- Se logró que los bloques se depositaran correctamente en el `CodingAreaPanel` al soltar el ratón.

- *Trabajo pendiente identificado:* Se detectó la necesidad de implementar la lógica para eliminar un bloque si, al soltarlo, no aterriza dentro de un área válida (como el `CodingAreaPanel`), replicando el comportamiento de Scratch.
- **Reevaluación de la Creación de Vistas de Bloques (Descubrimiento sobre UBlockly/Prefabs):**
 - Durante el análisis para resolver problemas de layout y conexiones distorsionadas, se descubrió que el enfoque óptimo (y el utilizado por el UBlockly original) es la creación de **prefabs preconfigurados en el editor de Unity** para cada tipo de bloque. Estos prefabs ya contienen la jerarquía correcta de componentes **View** (hijos visuales).
 - Este hallazgo llevó a un **replanteamiento parcial de la estrategia de instanciación de vistas**, enfocándose más en clonar y configurar estos prefabs en lugar de construir la jerarquía visual de cada bloque completamente desde cero en tiempo de ejecución. El objetivo era que la estructura del prefab dictara la disposición interna, evitando errores de posicionamiento derivados de contenedores dinámicos.
- **Problemas y Avances en el Snap Visual:**
 - Se logró la detección lógica de conexiones `NextConnection` y `PreviousConnection` y su vinculación en el modelo.
 - *Desafío persistente identificado:* Al realizar el snap visual, se observaron distorsiones en la imagen del bloque o la destrucción inesperada de la `BlockView`. Se identificó la necesidad de una revisión más profunda del sistema de layout manual (`BaseView.UpdateLayout()`) y cómo interactúa con las propiedades del `RectTransform` al re-parentar y ajustar tamaños tras una conexión.
- **Preparación Inicial de la Escena de Simulación:**
 - Se comenzó a configurar la escena 3D en Unity donde el robot operaría, definiendo el entorno básico y la cámara.
- **Hito Alcanzado/Resultado del Sprint:** Se logró una mejora significativa en el arrastre y posicionamiento inicial de los bloques en el área de codificación. Se identificaron y comenzaron a abordar los complejos desafíos del snap visual y la correcta gestión de coordenadas entre paneles. El descubrimiento sobre el uso óptimo de prefabs de UBlockly marcó un punto de inflexión para la estrategia de representación visual.

Se dispone de una herramienta de depuración de conexiones. La escena básica para la simulación del robot está iniciada.

Sprint 6: Implementación de Interacción Drag & Drop y Pruebas Iniciales del Prototipo de Simulación

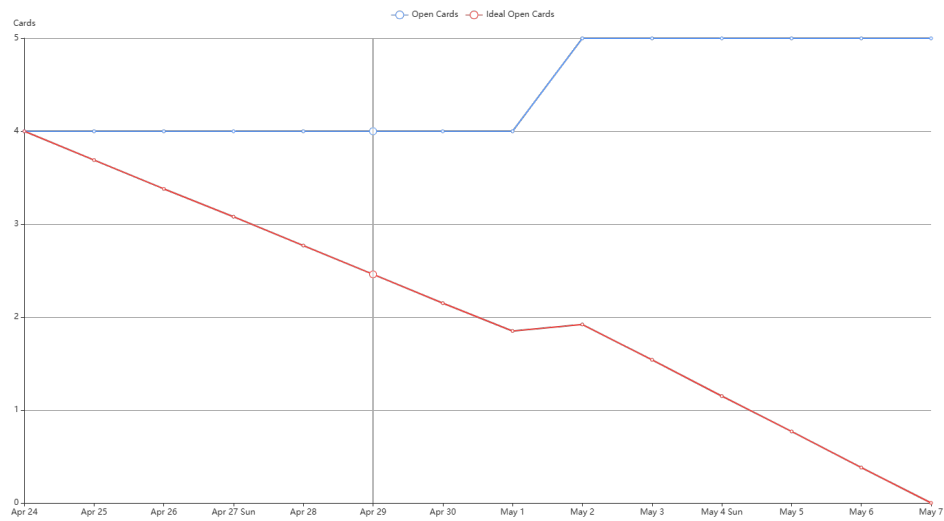


Figura A.15: Burndown sprint 6

- **Duración:** Del 24 de abril de 2025 al 29 de mayo de 2025.
- **Objetivo Principal:** Lograr una funcionalidad de arrastrar y soltar (*drag and drop*) robusta para los bloques, incluyendo la conexión (snap) visual y lógica entre ellos. Comenzar las pruebas del prototipo con la secuencia bandera verde seguida de mover 10 pasos para validar el flujo completo hasta la simulación del movimiento del robot.
- **Contexto y Desafíos:** Este sprint se enfrentó directamente a los desafíos de layout y posicionamiento identificados en el sprint anterior, particularmente en cómo las vistas de los bloques (**BlockView**) interactúan dentro de los diferentes paneles de Unity UI (Toolbox, DragLayer, Área de Codificación) y sus respectivos sistemas de coordenadas. La correcta implementación del snap visual y lógico era crítica. Adicionalmente, un contratiempo personal (una afección médica ocular

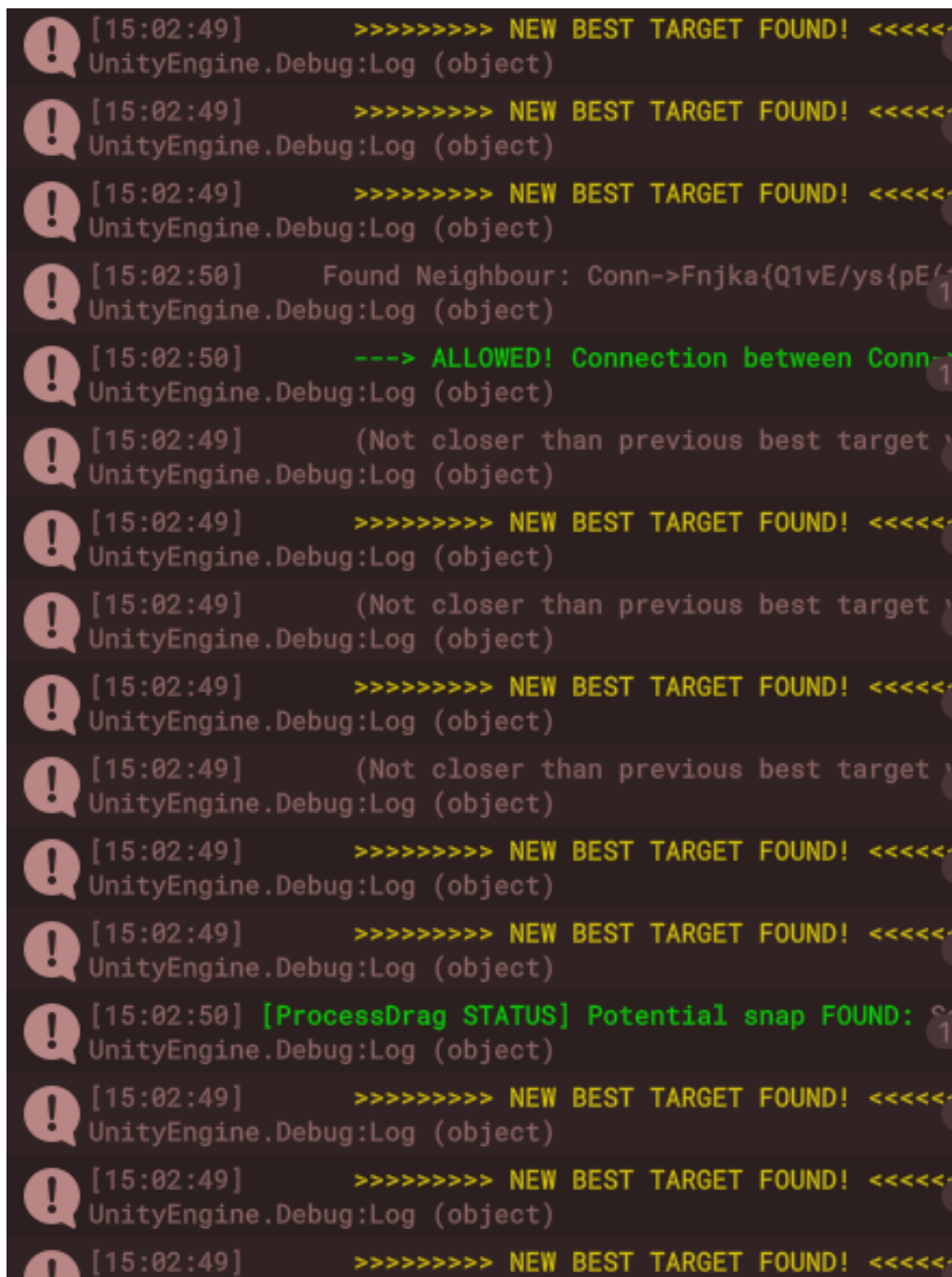


Figura A.14: Depuración de las conexiones desde la consola de unity cuando se ejecuta el programa.

provocada por un episodio de alergia intenso) impactó la capacidad de trabajo y extendió la duración de esta fase más allá de lo inicialmente previsto.

■ **Tareas Clave Realizadas y Resultados:**

• **Refinamiento del Sistema de Conexiones (Drag & Drop):**

- Se revisó y ajustó la lógica para la detección de posiciones cercanas entre los puntos de conexión de los bloques (`Connection_next`, `Connection_prev`, `Connection_output`), utilizando los *prefabs* de bloques diseñados y sus `ConnectionViews`.
- Se abordaron los problemas de conversión entre sistemas de coordenadas globales (pantalla) y locales (Canvas de cada panel) para el correcto posicionamiento y seguimiento de los bloques durante el arrastre.
- Se implementó la funcionalidad de *highlighting* (resaltado visual tipo sombra, similar a Scratch) en la conexión candidata cuando el bloque arrastrado se aproxima a un punto de snap válido. Inicialmente, la sombra no se posicionaba correctamente, pero tras ajustes en cómo se calcula su posición relativa al bloque objetivo, se logró el comportamiento deseado.
- Se logró el snap inicial entre dos bloques (ej. `event_whenflagclicked` y `motion_movesteps`), donde el bloque inferior se unía correctamente al bloque superior.

• **Implementación de Apilamiento de Bloques (Stacking) y Lógica de Bumping:**

- Se trabajó en extender la funcionalidad de "snap para permitir el apilamiento de múltiples bloques secuenciales (más de dos). Inicialmente, las conexiones `PreviousConnection` y `NextConnection` presentaban problemas al intentar conectar un tercer bloque a una pila ya existente.
- Se identificó y corrigió un error crítico (una referencia nula a la cámara en el sistema *Screen Space Overlay* de Unity UI) que impedía el correcto funcionamiento del snap visual en pilas más largas.
- Se programó e integró la lógica de bumping: cuando un bloque se inserta en medio de una pila ya conectada, los bloques inferiores se desplazan correctamente para hacer espacio, y

las conexiones se rehacen de forma adecuada. Se depuró la obtención de la propiedad `Location` en las `ConnectionModel`, crucial para esta funcionalidad.

- Se validó el apilamiento con múltiples bloques `motion_movesteps` bajo un bloque `event_whenflagclicked`.

- **Desarrollo de la Simulación y Movimiento del Robot:**

- Se configuró la escena 3D con el fondo y el posicionamiento inicial del robot virtual.
- **Desarrollo del Controlador del Robot (`RobotBehaviour`):** Se creó (o consolidó) el script `RobotBehaviour` encargado de recibir órdenes de movimiento (ej. mover N unidades) y ejecutar las transformaciones y animaciones correspondientes del `GameObject` del robot en la escena 3D, utilizando corrutinas de Unity para lograr movimientos suaves y controlados en el tiempo.
- **Implementación del Intérprete de Movimiento y Eventos:** Se desarrollaron y/o adaptaron los intérpretes específicos (`Cmdtors`) para los bloques clave del prototipo:
 - ◇ El `MotionBlockInterpreter` fue implementado para procesar el bloque `motion_movesteps`, extrayendo el parámetro de distancia y traduciéndolo en una orden para el `BlockObserver`.
 - ◇ Se implementó la lógica de interpretación para el bloque `event_whenflagclicked` para que funcionara como punto de inicio de la ejecución del programa.
- **Integración del Sistema de Observación y Comunicación (`BlockObserver`):** Se refinó el `BlockObserver` para que actuara como el puente crucial entre el sistema de ejecución de bloques (los `Cmdtors`) y el `RobotBehaviour`. Esta clase es responsable de recibir las órdenes lógicas (ej. mover 10 pasos) y delegar la ejecución de la animación/movimiento al `RobotBehaviour`, gestionando además el estado de la simulación.
- **Pruebas de Traducción y Ejecución del Flujo Completo:** Se realizaron verificaciones exhaustivas para asegurar que la secuencia de bloques de evento y movimiento se traducía correctamente, a través de los intérpretes y el observador, en una secuencia de acciones lógicas y visuales ejecutables por el robot en el entorno 3D.

- Se integraron los botones de Bandera Verde y Stop en la `UICanvasView` para permitir al usuario controlar el inicio y la detención de la ejecución del programa y la transición al modo de simulación.
- Se implementó la lógica de cuenta atrás visual antes de iniciar el movimiento del robot, proporcionando un feedback claro al usuario sobre el inicio inminente de la simulación.
- *Desafío y Solución en la Simulación del Movimiento:* Inicialmente, se observaron problemas donde el robot se movía de forma errática (salía disparado) o no respetaba la secuencia de la cuenta atrás. Se abordó este desafío mediante:
 - ◇ Configuración del `Rigidbody` del robot como `isKinematic = true` para un control programático del movimiento, evitando interferencias no deseadas con el motor de físicas de Unity.
 - ◇ Depuración detallada de la cadena de ejecución de corrutinas entre `CSharpRunner`, `MotionBlockInterpreter`, `BlockObserver`, y `RobotBehaviour` para asegurar que la espera (`yield return`) se propagaba correctamente y que las acciones del robot solo comenzaban tras la finalización de la cuenta atrás y en respuesta directa a las instrucciones de los bloques interpretados.

- **Herramientas de Depuración Adicionales:**

- Se habilitó la funcionalidad de exportar la base de datos de conexiones (estado del `WorkspaceModel`) para facilitar la depuración y el análisis del estado interno del sistema durante las pruebas de conexión.

- **Hito Alcanzado/Resultado del Sprint:** Se logró una funcionalidad de *drag and drop* y snap robusta para los bloques del prototipo (`event_whenflagclicked` y `motion_movesteps`), incluyendo el apilamiento múltiple y la lógica de bumping. El robot virtual ejecuta los movimientos programados en la escena 3D tras una cuenta atrás, de forma controlada. El flujo principal programar-ejecutar-simular es funcional y demostrable. El sistema permite una exportación del estado de la base de datos de conexiones para depuración.

A.3. Estudio de Viabilidad

Viabilidad Económica

Dado que este proyecto se desarrolla en el marco de un Trabajo Fin de Grado, los costes económicos se limitan principalmente a los recursos de hardware y software ya disponibles por el estudiante y la universidad, y a servicios de bajo o nulo coste para desarrolladores individuales.

Costes de Personal

El desarrollo ha sido realizado por un único estudiante como parte de su formación académica. No se contemplan costes salariales directos asociados al desarrollo del prototipo. El tiempo dedicado se considera inversión en aprendizaje y cumplimiento de los requisitos del TFG. La labor de tutorización por parte del profesor D. Carlos López Nozal se enmarca dentro de sus funciones docentes en la Universidad de Burgos.

Costes de Hardware

- **Equipo de Desarrollo:** Se ha utilizado un ordenador personal estándar con capacidad suficiente para ejecutar Unity, Blender y las herramientas de desarrollo.
- **Dispositivo de Prueba (Tablet Android):** Para las pruebas de portabilidad y usabilidad, se ha utilizado una tablet Android genérica.
- **Coste estimado de amortización de hardware (simulado):** Si se tuviera que estimar un coste de amortización para un proyecto independiente, se podría considerar un valor simbólico, pero para este TFG, los costes directos son nulos.

Costes de Software y Servicios

- **Unity 6 (Personal Edition):** Licencia gratuita para estudiantes y desarrolladores individuales.
- **C# y Visual Studio Community:** Herramientas gratuitas.
- **UBlockly:** Repositorio de código abierto, sin coste de licencia.
- **Blender:** Software de modelado 3D de código abierto y gratuito.

- **LEGO Digital Designer / Studio 2.0:** Software gratuito de LEGO para diseño.
- **GitHub:** Plan gratuito para repositorios públicos o privados (si aplica cuenta de estudiante).
- **Zube:** Ofrece planes gratuitos o de bajo coste para pequeños equipos o uso individual en proyectos de código abierto. Se ha utilizado bajo estas condiciones o como herramienta de prueba.
- **PlayerPrefs (Unity):** Almacenamiento local integrado, sin coste adicional.

En resumen, los costes económicos directos para el desarrollo del prototipo han sido **mínimos o nulos**, aprovechando licencias educativas, software de código abierto y recursos preexistentes.

Viabilidad Legal y Ética

Licencias de Software

- **Unity Personal Edition:** Su licencia permite el desarrollo y distribución de proyectos siempre que no se superen ciertos umbrales de ingresos o financiación, lo cual no aplica a este TFG.
- **UBlockly:** Generalmente distribuido bajo una licencia permisiva de código abierto (Apache 2.0). Se ha respetado la licencia original al adaptar el código, manteniendo las atribuciones correspondientes si fueran necesarias (aunque en este caso, la adaptación es para un TFG y no para redistribución comercial del UBlockly original).
- **Otros componentes (Blender, etc.):** Se han utilizado bajo sus respectivas licencias de código abierto.

Propiedad Intelectual

El código fuente desarrollado específicamente para este TFG, así como los diseños y modelos 3D originales creados, son propiedad intelectual del autor (Luis Ignacio de Luna Gómez) y de la Universidad de Burgos, según la normativa aplicable a Trabajos Fin de Grado. La adaptación del código de UBlockly se ha realizado con el fin de crear una nueva obra derivada para un propósito académico y de investigación.

Privacidad de Datos

El prototipo actual, al utilizar `PlayerPrefs` para el almacenamiento local de los programas, no recopila ni transmite datos personales del usuario a servidores externos. Por lo tanto, los requisitos de normativas de protección de datos como GDPR son mínimos en esta fase. Si en futuras líneas de trabajo se integrasen servicios en la nube que manejen datos de usuario, sería necesario realizar un análisis exhaustivo y aplicar las políticas de privacidad correspondientes.

Consideraciones Éticas

El proyecto tiene un fin educativo y busca promover el acceso a la tecnología. Se ha evitado el uso de assets o recursos con derechos de autor restrictivos sin la debida licencia. El diseño de la interfaz y la simulación buscan ser inclusivos y no discriminatorios.

Viabilidad Técnica

La viabilidad técnica del prototipo se ha demostrado mediante su implementación funcional.

- **Tecnologías Maduras:** Unity y C# son tecnologías robustas y ampliamente documentadas, lo que ha facilitado el desarrollo.
- **Adaptabilidad de UBlockly:** A pesar de no estar diseñado para replicar Scratch directamente, la arquitectura de UBlockly ha demostrado ser lo suficientemente flexible para ser adaptada a los requisitos del proyecto mediante la implementación de un patrón MVC y la personalización de vistas y controladores.
- **Limitaciones del Prototipo:** El prototipo actual implementa un conjunto limitado de bloques y funcionalidades. La viabilidad de expandirlo a un sistema completo con todas las características de Scratch dependería de un esfuerzo de desarrollo continuado, pero las bases técnicas establecidas son sólidas.
- **Rendimiento:** El prototipo, con su alcance actual, funciona de manera fluida en un equipo de desarrollo estándar y se espera un rendimiento aceptable en tablets Android de gama media, aunque esto requeriría pruebas específicas en la plataforma objetivo como parte de una línea futura.

En conclusión, el proyecto se considera viable tanto económica como legal y técnicamente dentro del contexto de un Trabajo Fin de Grado enfocado en el desarrollo de un prototipo funcional.

Apéndice B

Especificación de Requisitos

B.1. Introducción

A continuación se detalla la especificación de requisitos del proyecto Laboratorio Virtual de Programación y Robótica. Una adecuada definición y gestión de los requisitos es fundamental en ingeniería de software para asegurar que el sistema desarrollado satisfaga las necesidades y expectativas de los usuarios, así como para guiar las fases posteriores de diseño e implementación. Dada la naturaleza de prototipo del presente Trabajo Fin de Grado, esta especificación se centrará en los requisitos que guían la funcionalidad actual del prototipo, así como aquellos objetivos clave que sentarán las bases para futuras expansiones. Los requisitos se clasifican en funcionales (qué debe hacer el sistema) y no funcionales (cómo debe funcionar, relativo a la calidad).

B.2. Objetivos Generales del Proyecto

Los objetivos generales que guían el desarrollo de este laboratorio virtual han sido definidos previamente en el Capítulo 2 de esta memoria. Estos objetivos, que se persiguen tanto a nivel funcional como no funcional, se sintetizan a continuación:

- **Desarrollar un prototipo de laboratorio virtual de programación y robótica** que permita a los estudiantes aprender programación mediante bloques.

- **Replicar la experiencia de usuario y la lógica fundamental de Scratch** en un entorno Unity.
- **Integrar el sistema de bloques con una simulación 3D en Unity** para la visualización de acciones del robot.
- **Implementar una simulación básica de un robot LEGO WeDo 2.0** en el entorno virtual.
- **Garantizar la accesibilidad y portabilidad del prototipo** a plataformas Android (Tablets).
- **Investigar y analizar plataformas existentes** para informar futuras mejoras.
- **Adquirir conocimientos técnicos** en Unity y C# en el contexto del desarrollo de programación visual y simulación.

B.3. Catálogo de Requisitos

El siguiente catálogo enumera los requisitos clave del prototipo. Se organizan por tipo y se proporciona un identificador único (RF para Requisito Funcional, RNF para Requisito No Funcional), una descripción concisa, y su prioridad dentro del alcance actual del proyecto (Alta para funcionalidades esenciales del MVP, Media para deseables en el prototipo, Baja para futuras o de mejora).

Requisitos Funcionales (RF)

Estos requisitos describen las funcionalidades específicas que el laboratorio virtual debe proporcionar al usuario.

1. **RF-001: Gestión de la Interfaz Principal.** El sistema debe presentar una interfaz de usuario principal que incluya paneles para la selección de categorías de bloques, el listado de bloques disponibles y el área de codificación. **[Prioridad: Alta]**
2. **RF-002: Selección de Categorías de Bloques.** El usuario debe poder seleccionar una categoría de bloques para visualizar los bloques disponibles. **[Prioridad: Alta]**

3. **RF-003: Arrastre de Bloques.** El usuario debe poder seleccionar un bloque de la lista de categorías y arrastrarlo hasta el área de codificación. **[Prioridad: Alta]**
4. **RF-004: Conexión de Bloques.** El sistema debe permitir la unión visual (con efecto de snap) y lógica de bloques compatibles mediante arrastrar y soltar. **[Prioridad: Alta]**
5. **RF-005: Desconexión de Bloques.** El usuario debe poder separar bloques previamente conectados. **[Prioridad: Media]**
6. **RF-006: Edición de Campos de Bloques.** El sistema debe permitir al usuario editar valores en campos específicos de los bloques (ej. números, texto). **[Prioridad: Alta]**
7. **RF-007: Ejecución del Programa.** El usuario debe poder iniciar la ejecución de la secuencia de bloques mediante una acción específica (ej. click en bandera verde). **[Prioridad: Alta]**
8. **RF-008: Detención del Programa.** El usuario debe poder detener la ejecución del programa en cualquier momento (ej. click en botón de stop). **[Prioridad: Alta]**
9. **RF-009: Visualización del Robot y Simulación.** El sistema debe mostrar un modelo 3D de un robot LEGO WeDo 2.0 en un entorno de simulación. **[Prioridad: Alta]**
10. **RF-010: Ejecución de Movimiento del Robot.** El robot simulado debe ejecutar las acciones de movimiento (ej. mover 10 pasos) programadas con bloques de forma visible en el entorno 3D. **[Prioridad: Alta]**
11. **RF-011: Feedback Visual de Ejecución.** El sistema debe proporcionar feedback visual y textual (ej. mensajes, cuenta atrás, resaltado de bloque en ejecución) sobre el estado del programa y las acciones del robot. **[Prioridad: Alta]**
12. **RF-012: Persistencia del Trabajo.** El usuario debe poder guardar y cargar el estado de sus programas de bloques para continuar editándolos en futuras sesiones. **[Prioridad: Media]**
13. **RF-013: Carga de Definiciones de Bloques.** El sistema debe cargar las definiciones estructurales y visuales de los bloques (tipo, categoría, color, campos, conexiones permitidas) desde fuentes externas

(archivos XML), permitiendo la configuración y expansión de la librería de bloques disponibles sin necesidad de modificar el código fuente principal de la aplicación. **[Prioridad: Alta]**

14. **RF-014: Interpretación Bloque-Acción.** Para cada bloque con comportamiento ejecutable, el sistema debe disponer de un intérprete específico capaz de traducir la lógica y los parámetros del bloque en una acción o secuencia de acciones programáticas en el entorno Unity (ej. convertir el valor 10 del bloque mover 10 pasos en una orden de desplazamiento para el robot). **[Prioridad: Alta]**
15. **RF-015: Gestión Lógica del Espacio de Trabajo.** El sistema debe mantener un modelo lógico del espacio de trabajo que permita al usuario añadir, eliminar y reorganizar bloques. Este modelo debe gestionar la lista de bloques principales y servir como la fuente autoritativa del estado del programa. **[Prioridad: Alta]**
16. **RF-016: Descarte de Bloque.** El usuario debe poder eliminar bloques del espacio de trabajo, arrastrándolos fuera del área de codificación. La eliminación debe resultar en la correcta desvinculación y disposición del bloque y sus descendientes. **[Prioridad: Media]**.
17. **RF-017: Representación Visual de la Jerarquía de Bloques.** El sistema debe representar visualmente las pilas de bloques conectados secuencialmente y los bloques anidados dentro de entradas de tipo sentencia, apilándolos y indentándolos correctamente en la interfaz para reflejar la estructura del programa al estilo Scratch. **[Prioridad: Alta]**

Requisitos No Funcionales (RNF)

Estos requisitos describen las cualidades del sistema que garantizan un buen funcionamiento y una experiencia de usuario óptima.

1. **RNF-001: Rendimiento de la Interfaz.** La interfaz de programación visual debe responder a las interacciones del usuario (arrastre, conexión) con una latencia mínima, garantizando una experiencia fluida. **[Prioridad: Alta]**
2. **RNF-002: Consistencia Visual.** El diseño visual de los bloques y la interfaz debe ser consistente con la estética de Scratch. **[Prioridad: Alta]**

3. **RNF-003: Accesibilidad.** El sistema debe ser capaz de ejecutarse en dispositivos Android tipo tablet con hardware común, sin requerimientos técnicos elevados. **[Prioridad: Alta]**
4. **RNF-004: Precisión de la Simulación.** El movimiento y el comportamiento del robot simulado deben ser perceptibles y acordes con la lógica programada en los bloques. **[Prioridad: Alta]**
5. **RNF-005: Mantenibilidad.** La arquitectura del software debe facilitar la futura modificación y extensión del sistema de bloques y la lógica del robot. **[Prioridad: Alta]**
6. **RNF-006: Confiabilidad.** El sistema no debe experimentar caídas o bloqueos inesperados durante la interacción normal o la ejecución de programas. **[Prioridad: Media]**
7. **RNF-007: Modularidad del Código.** La arquitectura del software del sistema debe estar organizada siguiendo el patrón Modelo-Vista-Controlador (MVC), promoviendo una clara separación de responsabilidades. Los componentes dentro de cada capa (Modelo, Vista y Controlador) deben diseñarse con principios de *bajo acoplamiento* (interdependencia mínima entre módulos) y *alta cohesión* (elementos dentro de un módulo con funcionalidades estrechamente relacionadas). Esta estructura facilitará la comprensión del código, la depuración, la modificación aislada de componentes y el mantenimiento general del sistema. **[Prioridad: Alta]**
8. **RNF-008: Extensibilidad para Nuevos Bloques.** El diseño del sistema debe permitir la adición de nuevos tipos de bloques de programación con un impacto mínimo y localizado en el código fuente existente. La incorporación de un nuevo bloque debería, idealmente, implicar la creación de:
 - Una definición de bloque.
 - Una clase para su modelo lógico .
 - Un prefab para su representación visual en Unity UI .
 - Una clase intérprete si el bloque es ejecutable.

Estos nuevos componentes deben poder integrarse sin requerir modificaciones extensas en el núcleo del sistema o los controladores principales. **[Prioridad: Alta]**

9. **RNF-009: Usabilidad y Retroalimentación Visual Clara.** El sistema debe proporcionar indicaciones visuales claras e intuitivas al usuario durante las interacciones clave para facilitar su uso y comprensión:

- **Resaltado de Conexiones Válidas:** Durante la operación de arrastre de un bloque, las posibles conexiones compatibles con otros bloques deben resaltarse visualmente (ej. mediante un cambio de color o una superposición gráfica en el punto de snap) para guiar al usuario hacia una unión correcta. [Prioridad: Alta]
- **Retroalimentación ante Conexiones Inválidas:** Si el usuario intenta realizar una conexión entre bloques incompatibles, el sistema debería, en la medida de lo posible, indicar visualmente que la conexión no es permitida. [Prioridad: Media]
- **Indicación de Ejecución:** El bloque que se está ejecutando actualmente debe ser visualmente distinguible para que el usuario pueda seguir el flujo del programa. [Prioridad: Alta]

B.4. Especificación de requisitos

Actores del Sistema

Un actor representa un rol desempeñado por un usuario humano, un dispositivo hardware o incluso otro sistema software que interactúa con el sistema. Para el presente prototipo, se identifican los siguientes actores principales:

Casos de uso

A continuación, se detallan los casos de uso (CU) que describen las interacciones clave entre los actores y el sistema. Cada caso de uso se presentará en una tabla siguiendo un formato estándar.

CU01: Gestionar Interfaz de Programación

Actor A01: Estudiante (Usuario Principal)	
Descripción:	Individuo (generalmente un niño, adolescente o persona en formación) que utiliza el laboratorio virtual para aprender conceptos de programación visual y robótica mediante la creación y ejecución de programas con bloques.
Tipo:	Humano.
Objetivo Principal:	Crear programas de bloques para controlar un robot virtual y observar su comportamiento en un entorno de simulación 3D, con el fin de desarrollar habilidades de pensamiento computacional y lógica de programación.
Relaciones con Casos de Uso Principales:	■ Gestionar la interfaz de programación (CU01 - Ver apartado B.4). ■ Construir programas con bloques (CU02 - Ver apartado ??). ■ Ejecutar y simular programas (CU03 - Ver apartado ??). ■ Gestionar proyectos de bloques (CU04 - Ver apartado ??).

Tabla B.1: Descripción del Actor A01 - Estudiante.

Caso de Uso ID:	CU01
Nombre del Caso de Uso:	Gestionar Interfaz de Programación
Versión:	1.0
Autor:	Luis Ignacio de Luna Gómez
Actor(es) Principal(es):	A01 - Estudiante
Requisitos Funcionales Asociados:	RF-001, RF-002
Descripción Concisa:	El estudiante interactúa con la interfaz principal del laboratorio virtual para acceder a las herramientas de programación.
Precondiciones:	■ El sistema ha sido iniciado correctamente. ■ La interfaz principal es visible.
Flujo Principal de Eventos (Acciones):	1. El Estudiante visualiza los paneles principales: panel de categorías de bloques, panel de listado de bloques (Toolbox), y área de codificación. 2. El Estudiante selecciona una categoría de bloques (ej. "Movimiento") en el panel de categorías. 3. El sistema actualiza el panel de listado de bloques para mostrar las plantillas de bloques correspondientes a la categoría seleccionada.
Postcondiciones:	■ El panel de listado de bloques muestra los bloques de la categoría seleccionada.
Excepciones (Flujos Alternativos):	■ E1: No hay bloques definidos para una categoría seleccionada (el sistema muestra un mensaje informativo o el panel vacío).
Importancia:	Alta
Notas Adicionales:	Este caso de uso es la base para la interacción con el sistema de programación.

Tabla B.2: CU01 - Gestionar Interfaz de Programación.

Apéndice C

Especificación de diseño

- C.1. Introducción
- C.2. Diseño de datos
- C.3. Diseño arquitectónico
- C.4. Diseño procedimental

Apéndice D

Documentación técnica de programación

- D.1. Introducción
- D.2. Estructura de directorios
- D.3. Manual del programador
- D.4. Compilación, instalación y ejecución del proyecto
- D.5. Pruebas del sistema

Apéndice E

Documentación de usuario

- E.1. Introducción
- E.2. Requisitos de usuarios
- E.3. Instalación
- E.4. Manual del usuario

Apéndice F

Anexo de sostenibilización curricular

F.1. Introducción

El presente anexo recoge una reflexión personal sobre cómo los principios de sostenibilidad se han integrado y abordado durante el desarrollo de este Trabajo Fin de Grado (TFG). Más allá de los aspectos puramente técnicos, el proyecto "Laboratorio Virtual de Programación y Robótica" ha ofrecido la oportunidad de explorar y aplicar una visión de la ingeniería informática como una disciplina al servicio del desarrollo sostenible. Se profundiza en las competencias de sostenibilidad adquiridas y su aplicación práctica, demostrando que las soluciones tecnológicas pueden ser catalizadores para un futuro más equitativo y responsable, en línea con las directrices de la CRUE [1]

F.2. Contribución del Proyecto a los Objetivos de Desarrollo Sostenible (ODS)

El proyecto se alinea de manera fundamental con varios de los Objetivos de Desarrollo Sostenible de la ONU, especialmente en lo que respecta al acceso equitativo a la educación y la reducción de las desigualdades.

ODS 4: Educación de Calidad

El **ODS 4: Educación de Calidad** busca garantizar una educación inclusiva, equitativa y de calidad, así como promover oportunidades de aprendizaje durante toda la vida para todos. El laboratorio virtual contribuye directamente a este objetivo al derribar barreras geográficas y económicas que limitan el acceso a la enseñanza de la programación y la robótica. La dependencia de laboratorios físicos bien equipados y kits de robótica específicos representa un obstáculo considerable para centros educativos con recursos limitados o en zonas remotas. Al ofrecer una alternativa de simulación en Unity, se democratiza el acceso a la educación STEAM (Ciencia, Tecnología, Ingeniería, Arte y Matemáticas), permitiendo a niños y adolescentes experimentar con la programación de robots sin necesidad de adquirir costosos materiales o infraestructura. Esto promueve la inclusión de un mayor número de estudiantes en disciplinas de alto valor añadido, fomentando el desarrollo del pensamiento computacional y las habilidades digitales cruciales para el siglo XXI, asegurando que el origen socioeconómico no sea un factor determinante en el acceso a la calidad educativa.

ODS 10: Reducción de las Desigualdades

De forma inherente, el proyecto aborda el **ODS 10: Reducción de las Desigualdades**, el cual aspira a reducir la desigualdad dentro y entre los países, facilitando la inclusión social, económica y política de todas las personas. El laboratorio virtual disminuye la brecha de acceso a la tecnología y al conocimiento especializado que a menudo existe en contextos de vulnerabilidad. Al priorizar el desarrollo para dispositivos Android tipo tablet, se garantiza que la plataforma sea accesible en una gama más amplia de hardware de bajo coste, extendiendo así la oportunidad de aprender programación y robótica a segmentos de la población que tradicionalmente quedarían excluidos. La gratuidad de las herramientas empleadas para su desarrollo (Unity Personal, Csharp, UBlockly de código abierto, Git/GitHub) reduce drásticamente el coste de implementación y replicación de esta solución, permitiendo que organizaciones educativas, comunidades y familias puedan acceder a recursos didácticos de vanguardia sin que la inversión económica sea un impedimento. Este enfoque minimiza las disparidades en las oportunidades de aprendizaje, contribuyendo a una sociedad más equitativa e inclusiva.

F.3. Competencias de Sostenibilidad Adquiridas y Aplicadas

Durante el desarrollo de este TFG, he tenido la oportunidad de aplicar y fortalecer diversas competencias clave relacionadas con la sostenibilidad, trascendiendo el ámbito técnico y proyectándose hacia un impacto social y medioambiental.

Pensamiento Crítico y Sistémico

El desafío de crear un laboratorio virtual que sea accesible y educativo ha requerido un profundo pensamiento crítico y sistémico. Comprender las interconexiones entre la falta de recursos, las desigualdades educativas y la solución tecnológica implicaba analizar cómo cada componente del diseño (desde la elección de Unity y UBlockly hasta la simplificación del modelo de robot LEGO WeDo 2.0) impactaría en la accesibilidad global y en la viabilidad a largo plazo. Se ha procurado optimizar los algoritmos de simulación para un menor consumo de recursos, y el diseño de la interfaz para la usabilidad en pantallas pequeñas, evidenciando una perspectiva integral sobre cómo la tecnología se inserta en un contexto social y económico.

Diseño Orientado al Impacto Social y a Largo Plazo

La reflexión sobre la sostenibilidad ha influido significativamente en las decisiones de diseño. En lugar de desarrollar una solución ad-hoc, se ha priorizado la creación de un sistema extensible y modular que pueda evolucionar. La elección de UBlockly y la arquitectura MVC responden a la necesidad de construir una base que pueda adaptarse a nuevos tipos de bloques, modelos de robot y plataformas (como futuras versiones web o de realidad virtual) sin tener que reinventar la rueda, prolongando la vida útil del software y maximizando su potencial impacto educativo a largo plazo. Esta visión a largo plazo minimiza el despilfarro de recursos de desarrollo y facilita la resiliencia del proyecto ante futuros cambios tecnológicos o necesidades pedagógicas.

Gestión Ética de la Información y Responsabilidad Profesional

La utilización de herramientas de código abierto como UBlockly y la contribución implícita a la comunidad de conocimiento libre forman parte

de una gestión responsable y ética. Se ha prestado atención a la calidad y claridad del código, así como a la documentación técnica, para asegurar que el proyecto pueda ser mantenido y extendido por otros desarrolladores, fomentando la colaboración y el intercambio de conocimientos. Además, la conciencia sobre la importancia de la privacidad de los usuarios (aunque este prototipo no gestione datos sensibles, la mentalidad de diseñar para la seguridad futura es clave) y la integridad del proceso de desarrollo han sido principios rectores que han consolidado mi comprensión de la responsabilidad que implica el diseño de sistemas informáticos.

F.4. Reflexión Final

El desarrollo de este TFG ha sido una experiencia transformadora que ha trascendido la mera aplicación de conocimientos técnicos en programación. Me ha permitido conectar la teoría de la ingeniería informática con desafíos sociales concretos y con los principios universales de la sostenibilidad. La convicción de que la tecnología bien diseñada, con un propósito claro y un enfoque ético, tiene un potencial inmenso para contribuir a un mundo más justo y equitativo se ha reforzado enormemente. Este proyecto es una muestra de cómo las competencias técnicas y de sostenibilidad se entrelazan para dar vida a soluciones innovadoras que abren nuevas oportunidades educativas y contribuyen al bienestar global. Este trabajo no es solo un ejercicio académico, sino el inicio de una vocación por construir un futuro donde la tecnología sirva como un verdadero motor de cambio sostenible.

Bibliografía

- [1] CRUE Universidades Españolas. Directrices para la introducción de la sostenibilidad en el currículo universitario. https://www.crue.org/wp-content/uploads/2020/02/Directrices_Sostenibilidad_Crue2012.pdf, 2012. [En línea; consultado el 30-mayo-2025].
- [2] Zube Inc. Zube - project development managemet for agile development. <https://zube.io>, 2025.